MANUALE UTENTE QT-SERVER

-Versione Estesa-

Indice dei contenuti:

1) Introduzione
   1.1) Algoritmo QT
   1.2) Funzionamento algoritmo QT
   1.3) Estensione
2) Funzionamento Software
   2.1) Get Ready
   2.2) Utilizzo del Software

1 Introduzione

Il presente manuale descrive il funzionamento del software server sviluppato per il progetto del corso ***Metodi Avanzati di Programmazione***. Il server ha il compito di ricevere le richieste provenienti dai client, contenenti il nome della tabella da analizzare e il valore del raggio da utilizzare. Una volta ricevuta la singola richiesta, accede al database fornito dalla docente, estrae la tabella indicata ed esegue l'**algoritmo QT (Quality Threshold Clustering)**. Al termine dell'elaborazione, i risultati dei cluster generati vengono restituiti, centralizzando così l'esecuzione dell'algoritmo e garantendo un accesso diretto ed efficiente al database, mentre al client resta il solo compito di fornire i parametri necessari.

Inoltre, ad ogni esecuzione l'algoritmo salva il Cluster set prodotto dall'algoritmo QT in un file dedicato. Tale file può essere recuperato e inviato su richiesta al Client, consentendo di riutilizzare e consultare i risultati delle elaborazioni precedenti.

1.1 Algoritmo QT

L'algoritmo QT è uno strumento che serve a raggruppare i dati in insiemi ordinati e coerenti. In pratica, prende i dati in ingresso e li **organizza in gruppi**, chiamati *cluster*, all'interno dei quali gli elementi sono simili tra loro.

Ogni volta che l'algoritmo viene eseguito, produce un risultato che mostra come i dati si distribuiscono nei vari gruppi, facilitando così l'analisi e la comprensione delle informazioni. L'aspetto utile di questo approccio è che non è necessario decidere prima quanti gruppi creare: il sistema si occupa di costruirli automaticamente sulla base del **raggio** di definizione, garantendo che all'interno di ciascun gruppo ci sia coerenza tra i dati raccolti.

1.2 Funzionamento algoritmo QT

L'algoritmo QT (Quality Threshold Clustering) è un metodo di clustering che si basa sull'idea di formare gruppi di elementi omogenei tra loro, imponendo come vincolo una soglia di qualità definita dal diametro massimo consentito all'interno di ciascun cluster.

Il processo inizia considerando ogni elemento del dataset come punto di partenza e costruendo, attorno ad esso, il cluster più ampio possibile che rispetti la soglia prefissata. Una volta generati i cluster candidati, viene selezionato quello con il numero di elementi maggiore, che viene quindi fissato come centroide di quello specifico cluster all'interno della raccolta.

Gli elementi appartenenti a tale cluster vengono rimossi dal dataset e l'algoritmo ripete lo stesso procedimento sui dati rimanenti fino a quando non vi sono più elementi disponibili. In questo modo si ottiene un insieme di cluster compatti e coerenti, senza la necessità di specificare a priori il numero di gruppi da generare.

1.3 Estensione

In questa versione del software il server è stato aggiornato per ricevere in un unico messaggio tutte le informazioni necessarie all'elaborazione. Ciò significa che, invece di ricevere separatamente la scelta dell'operazione, il nome della tabella e il valore del raggio, il server ottiene questi tre dati insieme in **un'unica comunicazione** dal client.

Questa scelta è stata causata dall'implementazione del **form grafico** dal lato Client che prende queste informazioni contemporaneamente. Comunque questo approccio rende più immediata la gestione della richiesta e riduce il numero di interazioni necessarie tra client e server.

2 Funzionamento Software

In questa sezione vengono illustrate le modalità di utilizzo del software sviluppato e tutto ciò che è necessario per il suo corretto funzionamento.

2.1 Get Ready

Prima di avviare il software è necessario verificare che siano soddisfatti alcuni requisiti preliminari. L'utente deve innanzitutto assicurarsi che sulla macchina sia installato il **Java Development Kit (JDK)**, indispensabile per l'esecuzione dell'applicativo sviluppato in ambiente Java.

Inoltre, è importante che sulla macchina che ospita il **Server**, la **Porta 8080** sia libera poiché il server è configurato per utilizzare la suddetta porta. Se la porta risulta occupata da altri processi, il server non potrà avviarsi correttamente.

Una volta rispettati questi prerequisiti, l'utente deve aprire la **cartella del progetto** e accedere al percorso `ProgettoBase/QTServer/JAR-BAT`. Da qui è sufficiente eseguire il file batch **QTServer** con un doppio click, così da avviare il programma e poter proseguire con l'utilizzo delle sue funzionalità. All'interno della cartella **JAR-BAT** deve essere presente la cartella **file_salvataggio** in quanto all'interno di quest'ultima verranno salvati i file contenenti i Cluster Set ottenuti dall'algoritmo QT (se questa è assente allora il server funzionerà ma non riuscirà a salvare correttamente i cluster ottenuti e quindi non sarà possibile usufruire della seconda opzione, ovvero caricare i Cluster Set da file).

2.2 Utilizzo del Software

Dopo aver fatto doppio click sul file **.bat**, nella finestra a riga di comando viene mostrato un messaggio che indica che il **server è pronto ad accettare richieste** da parte di uno o più client. Subito dopo viene stampata a video il contenuto del **Server-Socket** creato, che il server utilizza per mettersi in ascolto delle connessioni. Fino a quando nessun client invia richieste, il server rimane in attesa.

```
QTServer started !!

The server is running and waiting for requests from clients.
|---[ The socket set up to listen for requests is: ServerSocket[addr=0.0.0.0/0.0.0.0,localport=8080] ]
|
```

Nel momento in cui un client tenta di connettersi, il programma segnala a video la creazione di un **socket dedicato** a quel client. Questo socket assume il ruolo di identificatore, permettendo di distinguere le attività di ciascun client connesso. La riga che segnala la creazione di un nuovo socket inizia sempre con il simbolo **°|->**, che permette di riconoscere facilmente l'apertura di una nuova connessione.

Dopo la creazione del socket dedicato, nella finestra a riga di comando vengono mostrate tutte le azioni eseguite da quel client. Ogni azione è preceduta dal simbolo **|->**, seguito dall'**identificatore** (ovvero il socket che rappresenta il client). Questa notazione è **indentata verso destra** rispetto al simbolo **°|->**, che invece indica la creazione del socket stesso. In questo modo, grazie all'indentazione, è possibile distinguere facilmente la fase di apertura di una nuova connessione dalle azioni compiute dai client già collegati.

```
°|-> Dedicated socket successfully created: Socket[addr=/127.0.0.1,port=60357,localport=8080]
  |-> Socket[addr=/127.0.0.1,port=60357,localport=8080]: Chose to load a table from the database
  |-> Socket[addr=/127.0.0.1,port=60357,localport=8080]: The table 'playtennis' has been successfully loaded
  |-> Socket[addr=/127.0.0.1,port=60357,localport=8080]: The QT algorithm was successfully executed and generated 3 Clusters
  |-> Socket[addr=/127.0.0.1,port=60357,localport=8080]: The Clusterset was saved in: file_salvataggio\playtennis_2.0.dat
```

Se una delle azioni provoca un errore, anche questo viene riportato a video utilizzando la stessa indentazione. L'errore appare quindi subito sotto la riga relativa all'azione che lo ha generato, sempre con il simbolo **|->** e con l'identificatore del socket interessato.

Ogni riga indentata **deve riportare** l'identificatore del client, poiché il server è **multi-thread** e può gestire più richieste contemporaneamente. In questo caso, può accadere che azioni appartenenti a client diversi vengano stampate una di seguito

all’altra, in quanto il server esegue le operazioni in parallelo e alterna l’elaborazione delle varie richieste provenienti da client diversi.

```
°|-> Dedicated socket successfully created: Socket[addr=/127.0.0.1,port=64710,localport=8080]
  |-> Socket[addr=/127.0.0.1,port=64710,localport=8080]: Chose to load a clusterset from a file
  |-> Socket[addr=/127.0.0.1,port=64710,localport=8080]: Loaded the cluster set from the file 'file_salvataggio\playtennis_3.0.dat'
  |-> Socket[addr=/127.0.0.1,port=60357,localport=8080]: The QT algorithm was successfully executed and generated 2 Clusters
  |-> Socket[addr=/127.0.0.1,port=60357,localport=8080]: The Clusterset was saved in: file_salvataggio\playtennis_3.0.dat
```

Autori:

- Francesco Lorusso
- Matteo Lupis